

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELGAVI -590 014



CSE STREAM

“OPERATING SYSTEM”

SEMESTER – III 'B' SEC (B.E)

Subject Code: BCS303

(BCS Scheme)



DEPARTMENT OF STUDIES IN COMPUTER SCIENCE AND ENGINEERING

**University BDT College of Engineering
(Constituent College of Visvesvaraya Technological University)
Davanagere-577002**

PROCESS SCHEDULING:

- Process scheduling is the core activity of the CPU scheduler in an Operating System.
- It decides which process should run next on the CPU to maximize efficiency.

Categories of Scheduling:

There are two categories of scheduling in the operating system.

1. Preemptive
2. Non-Preemptive

1. Preemptive:

- In this scheduling, the OS allocates a resource to the process for a fixed time interval.
- The process state changes from waiting to ready or ready to running state.

2. Non-Preemptive:

- In this, once the resource is allocated to a process, it cannot be changed.
- It only changes when process execution is completed, or process is moving to the waiting state.

Types of Process Schedulers:

S.No	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler process.	This uses CPU scheduling.	This process uses a swapping scheduler.
2.	Selects the process from the pool based on performance and resource availability.	Select a process from the ready queue for execution on the basis of multiple factors.	Finds the process to be swapped in and out.
3.	Executes less frequently.	Executes very frequently every time the process transitions to the running state.	Executes more frequently than the long term scheduler.
4.	Speed is lesser than short-term scheduler.	Fastest amongst the two.	Speed is in between the long-term and short-term scheduler.
5.	It controls the degree of multiprogramming.	It gives less control over how much multiprogramming is done.	It reduces the degree of multiprogramming.
6	It is almost absent or barely in the sharing process.	It is also minimal in the sharing process.	It is a part of the time-sharing process.

SCHEDULING CRITERIA:

- Scheduling criteria are metrics used to judge how good a CPU scheduling algorithm is.
- When comparing FCFS, SJF, RR, Priority etc., we evaluate them using these criteria.

1. CPU Utilization

- How busy the CPU is.
- Should be as high as possible (40–90% or more).
- Range of CPU utilization is in the range of 0 to 100.
- The goal is to maximize CPU utilization & minimize idle time.

Example:

- If CPU is idle for long time → poor scheduler.
- If a CPU runs for 80ms out of 100ms, the utilization is 80%.

2. Throughput

- Number of processes completed per unit time.
- Higher throughput = better performance.

Example:

- Completing 10 processes/min vs 5 processes/min.
- If a system completes 5 processes in 10 seconds, the throughput is 0.5 processes per second.

3. Turnaround Time

- Total time taken from submission to completion of a process.
- $Turnaround = Waiting + CPU\ time + I/O\ time + other\ delays$

Example:

- A student submits a program at 10:00 AM, it finishes at 10:10 AM → TT = 10 minutes.
- A process arrives at time = 2 ms and completes at time = 10 ms. (TAT = 10 – 2 = 8 ms.)

4. Waiting Time

- Time a process spends in the ready queue (not running).
- Not counting CPU or I/O time.

Example:

- Process waits 6 seconds in ready queue → $WT = 6 \text{ sec.}$
- A process has $TAT = 10 \text{ ms}$ and $CPU \text{ burst} = 6 \text{ ms.}$ ($WT = 10 - 6 = 4 \text{ ms.}$)

5□. Response Time

- Time between submission and first response (first time CPU gives output).
- Important in interactive systems.

Example:

- When you click a button, how quickly UI responds.
- A process arrives at time = 1 ms and gets CPU for the first time at time = 4 ms.
 $RT = 4 - 1 = 3 \text{ ms.}$

SCHEDULING ALGORITHMS:

Types of Scheduling Algorithms in OS

1. First Come First Serve (FCFS)
2. Shortest Job First (SJF)
3. Round Robin Scheduling(RR)
4. Priority Scheduling(PR)
5. Shortest Remaining Job First (SRJF)

NOTE: I HAVE EXPLAINED THE TOPIC AND PROBLEMS IN THE CLASS and LAB... REFER

THREAD SCHEDULING:

- Thread scheduling is the process by which the operating system decides which thread should get the CPU next.
- Since threads are the basic unit of CPU utilization, the CPU scheduler schedules threads, not processes.

Why Thread Scheduling is Important

- Threads share the same process resources.

- Faster context switching than processes.
- Improves CPU utilization and responsiveness.

Types of Thread Scheduling:

1□. User-Level Thread Scheduling

- Thread scheduling is handled by the user-level thread library
- OS sees the entire process as one unit
- Faster, but blocking system calls block all threads

Example: POSIX user threads

2□. Kernel-Level Thread Scheduling

- Thread scheduling is handled by the operating system
- OS schedules each thread individually
- If one thread blocks, others can continue

Example: Linux threads, Windows threads

Thread Scheduling Models

1. Many-to-One

- Many user threads → one kernel thread
- Blocking affects all threads

2. One-to-One

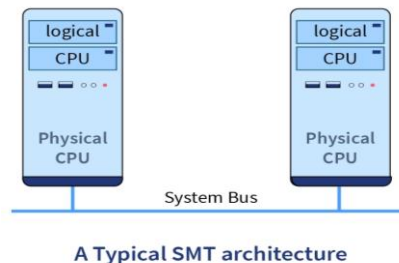
- Each user thread → one kernel thread
- True parallelism (used in Linux, Windows)

3. Many-to-Many

- Many user threads → many kernel threads
- Flexible & efficient

MULTIPLE-PROCESSOR SCHEDULING:

- Multiple-processor scheduling deals with how processes/threads are scheduled when more than one CPU is available in a system.
- Instead of one CPU, many CPUs execute tasks in parallel.

**Types of Multiprocessor Systems****1. Asymmetric Multiprocessing (AMP)**

- One master processor handles all scheduling decisions.
- Other processors execute only user processes.
- Simple to design, less flexible.

Example: Master CPU assigns tasks → Worker CPUs execute.

2. Symmetric Multiprocessing (SMP)

- All processors are equal
- Each CPU performs its own scheduling.
- Shared memory and ready queue.

Example: Modern Linux, Windows systems.

Scheduling Approaches in SMP:**i. Global Queue Scheduling**

- One common ready queue for all CPUs.
- Any free CPU picks the next process.
- Can cause cache inefficiency

ii. Per-Processor Queue Scheduling

- Each CPU has its own ready queue or private ready queue.
- Processes usually stay on the same CPU.
- Better cache performance

- Load imbalance possible

3. Load Balancing

- Ensures all CPUs are equally utilized.
- Load Balancing is the phenomenon of distributing workload so that the processors have an even workload in a symmetric multi-processor system.

Two techniques:

1. Push migration: OS moves processes from busy CPU to idle CPU
2. Pull migration: Idle CPU pulls work from busy CPU

4. Processor Affinity

- A process has an affinity for a processor on which it runs. This is called processor affinity.
- A process prefers to run on the same CPU where it ran before (better cache usage).

Types:

1. Soft affinity: The system has a rule of trying to keep running a process on the same processor but does not guarantee it. This is called soft affinity.
2. Hard affinity: Process is bound to a specific CPU. i.e each process can run only some of the processors.

5. Multicore Processors

- Multiple processor cores on a single physical chip.
- Each core appears as an independent processor to the OS.

6. Virtualization and Threading

Advantages of Multiple-Processor Scheduling

- Efficient Multitasking
- Resource Utilization
- Fault Tolerance
- Scalability

Dis-Advantages of Multiple-Processor Scheduling

- Increased Complexity
- Synchronization Overhead

- Software Compatibility
- Cost

BCS303

Chapter 3

PROCESS SYNCHRONIZATION

Introduction:

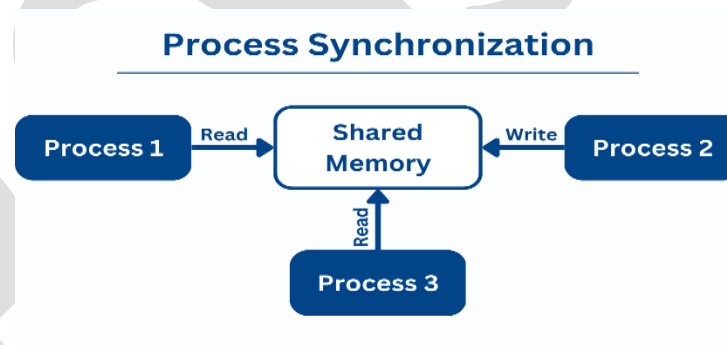
Independent and cooperative process:

- Independent: This type of process is not dependent on any process, they run individually.
- Cooperative: This type of process are dependent on each other. They share resources between each other.

Why process Synchronisation is Important?

Explained in class: Refer class notes (producer- consumer problem: Bounded Buffer)

- Process synchronization refers to the **method of coordinating the execution of processes** that share resources.
- Process Synchronisation **is much needed in Cooperative** process. As the process share the resource between each other.(Data consistency is maintained)
- Synchronization is about making sure that processes access shared resources in a **well-defined, predictable, and non-conflicting manner**.
- Synchronization ensures that only one process uses a shared resource at any given time, maintaining **data consistency and system stability**.



1. Independent Process: The execution of one process does not affect the execution of other processes.
2. Cooperative Process: A process that can affect or be affected by other processes executing in the system.

Points: Remember of Producer and Consumer Problem: Shared Memory (Buffer Region)

Two kind of buffers which is the bounded buffer and the unbounded buffer

Example of Producer and Consumer: Count = 10

Producer: • register1 = count; • register1 = register1 + 1; • count = register1;	Consumer: • register2 = count; • register2 = register2 - 1; • count = register2;
---	---

Example:

Time	User	Status	Code	Result
T1	Producer	executes	register1=counter;	register1=10
T2	Producer	executes	register1 = register1 + 1;	register1=11
T3	Consumer	executes	register2 = count;	register2=10
T4	Consumer	executes	register2 = register2 - 1;	register2=9
T5	Producer	executes	count=register1;	Count=11
T6	Consumer	executes	count=register2;	Count=9

The Final result will 9 or 10 or 11:

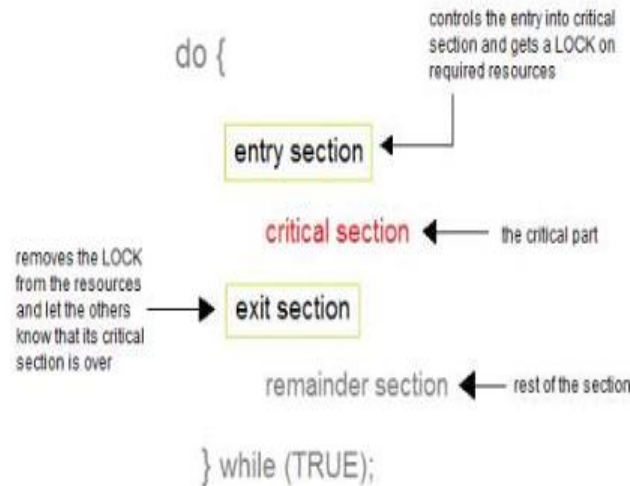
- If process synchronisation does not happen (Result: 9 or 11)
- If process synchronisation happens (Result: 10)

Race Condition

- A race condition occurs when two or more processes or threads access and modify shared data at the same time, and the final result depends on the order of execution.
- Race condition happens due to lack of synchronization
- It is solved using critical section, locks, semaphores, or mutex.

THE CRITICAL SECTION PROBLEM

- A Segment of code in the process.
- A part of code that can only be accessed by a single process at any moment is known as a critical section.
- This means that when a lot of programs want to access and change a single shared data, only one process will be allowed to change at any given moment.
- when one process is executing in its critical section, no other processes is allowed to execute in its critical section



1. Entry section: Then the section of code implementing this request is the.
2. Critical section: The section of code that will implement the request of the process to enter its critical session.
3. The critical section may be followed by an exit section.
4. The remaining code is the remainder section.

A solution to critical section problem must satisfy three key conditions:

- Mutual Exclusion: Only one process should be inside its critical section at any given time.
 - Only one process can be in the critical section at a time.
 - If P_1 is in CS, P_2 cannot enter CS
- Progress:
 - If no process is in CS, and some want to enter,
 - The decision of who enters next must be made without delay.
 - Only processes not in remainder section participate in the decision.
- Bounded Waiting: Every process should get a fair chance to enter its critical section; none should wait forever.
 - There must be a limit on how long a process waits.
 - Prevents starvation.
 - Every process gets a fair chance.
- Hardware Dependence: no dependence on hardware, Any hardware configuration should can help in avoiding critical section problem.

Note: Add the class notes points also

PETERSON'S PROBLEM:

- It is a classical software-based solution for critical section problem and to achieve process synchronisation.
- It is Humble Algorithm: Its given turn to opponent process.

In Peterson's solution, we have two shared variables that are used by the processes.

1. A Boolean Flag [2]: A Boolean array Flag. This Flag array represents which process is which process wants to enter into the critical solution. (2 -> True or False).
 2. int Turn: An integer variable Turn indicates the process number that is ready to enter into the critical section.
- Points:
 - Modern Computer system does not Supports Peterson problem solution. But the Algorithm description is used in software Design.
 - Peterson Solution is restricted to two processors which alternate execution between Critical section and remainder section.

For Process P1 (A)	For Process P2 (B)
<pre>do { Flag[A] = True; Turn = B; While (Flag[B] && Turn==[B]); Critical Section Flag[A] = false; remainder Section } While(True);</pre>	<pre>do { Flag[B] = True; Turn = A; While (Flag[A] && Turn == [A]); Critical Section Flag[B] = false; remainder Section } While(True);</pre>

Explanation: Explained in class: Add those points

Two process can't enter critical section concurrently.

While Condition: When both conditions are true, the process loops, can't enter critical section.

When any one condition becomes false, the process enters critical section.

Ex: P1 and P2 wants to enter critical section at a time. Example is explained in class

1. **Mutual Exclusion**
2. **Progress**
3. **Bounded Buffers**

BCS303